同一串位元，先問它代表什麼

硬體只收到有限長度的位元；**位元如何解讀、結果保留幾位、超出範圍如何處理**，才決定運算的意義。

問題	必須先固定	本章追蹤的結果
整數加減	位寬、有號或無號、溢位政策	結果位元、進位與溢位
整數乘除	暫存器寬度、每輪操作、商的取整方式	逐輪狀態、最後乘積或商餘數
浮點運算	格式、有效精度、捨入模式、例外環境	編碼、捨入結果與旗標

例如 `11111111` 可代表無號的 255，也可代表八位元二補數的 −1。位元沒有改變，解讀不同；後續的比較、除法與高位擴充也就不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種結果規則：截低位、飽和、捨入

固定寬度無法表示所有整數與實數。先算出數學上的精確值，再看指定的格式如何保存它。

八位元輸入 100+100	保存的結果	解讀
保留低八位	`11001000`	無號為 200；二補數為 −56
有號飽和	`01111111`	夾到有號上界 127
有號且檢查溢位	依指令處理例外	數學和 200 超出 [−128,127]

浮點數則依捨入模式選可表示的候選值，不是把整個實數當成整數直接截低位。格式、運算規則與時序是三個問題：**得到正確結果，不代表只需一個週期。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 先寫清楚運算模型

無號 n 位元數的範圍是 $[0, 2^n - 1]$；n 位元二補數的範圍是 $[-2^{n-1}, 2^{n-1} - 1]$。加減若只保留低 n 位，位元結果等於模 2^n 運算；有號的數學結果是否可表示，仍要另外判斷。

有限浮點輸入的基本運算則先求精確值，再依目標格式與捨入模式映回可表示集合：

$$y = \mathrm{round}(x \circ z).$$

此式描述一般有限值路徑；NaN、無窮大、除以零與超出範圍要套用另外的規則。整數 `0/0` 的行為依指令或語言而定；IEEE 浮點 `0/0` 屬 invalid，停用 trap 時通常回傳 quiet NaN。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 二補數：相同加法器，不同範圍判斷

n 位元二補數中，最高位帶負權重：

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

加法器逐位相加的規則與無號加法相同。減法利用 $a - b = a + \bar{b} + 1$，把 b 的每位反相、最低位帶入進位 1，就能共用加法器。

例如四位元 `0101+0011=1000`。無號解讀是 5+3=8；二補數解讀時，5 和 3 均合法，但數學和 8 超出上界 7，`1000` 反而表示 −8。**低位結果正確與有號結果可表示，必須分開檢查。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有號溢位：為什麼要比較符號？

一正一負相加，和介於兩個輸入之間，因此不會超出有號範圍。同號相加才可能超界；超界後保留低 n 位，結果符號翻轉。令輸入與結果的符號位為 s_a, s_b, s_r，則

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

減法的對應條件為 $ov = (s_a \oplus s_b) \wedge (s_a \oplus s_r)$：異號相減才可能超界，且結果符號與 a 相反。

八位元 `0x7f+1` 有有號溢位卻沒有最高位進位；`0xff+1` 有最高位進位卻沒有有號溢位。進位離開最高位判斷的是**無號加法是否超界**，不能直接當成有號溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐位核對：進入與離開符號位的進位

對八位元加法，令 c_7 進入符號位、c_8 離開符號位。有號溢位等價於 $ov = c_7 \oplus c_8$。

運算	低八位結果	c_7	c_8	有號溢位
`0x7f+0x01`	`10000000`	1	0	1
`0x80+0xff`	`01111111`	0	1	1
`0x80-0x01`	`01111111`	0	1	1
`0xff+0x01`	`00000000`	1	1	0

第三列實作為 `10000000+11111110+1`，最低位輸入進位為 1。符號位滿足 $s_r = s_a \oplus s_b \oplus c_7$，而其進位輸出為三輸入的多數決；代入同號與異號兩種情況，即得上述 XOR 判定。減法表中的 c_8 是加法器輸出，**不是一律等於 borrow（借位）**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加法器的等待時間與指令例外是兩件事

漣波進位加法器（ripple-carry adder）的高位結果要等低位進位，最長路徑隨位寬成長。前瞻進位或平行前綴結構，用更多邏輯與連線縮短依賴鏈；實際時脈仍須容納最慢可成立路徑及暫存器成本。

在本章傳統 MIPS32 案例，`add` 檢查有號溢位；發生例外時不能把錯誤結果當成正常寫回。`addu` 則保留模 2^{32} 的結果，不因有號超界而 trap。名稱中的 u 不表示加法器只能接收正數。

因此要分別回答：位元結果是什麼、指令允許它寫回嗎、硬體多久能產生結果。只看運算元的平均情況，不能證明最壞時序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

飽和與 SIMD：每個 lane 都有自己的邊界

飽和運算（saturating arithmetic）超界時夾到端點。例如有號八位元 100+100 得 127；保留低八位則得到 −56。音訊振幅可能需要飽和，雜湊或校驗碼的模運算則不能任意改用它。

SIMD 將一個暫存器切成多個 lane，分別做短整數運算。**進位不跨 lane 傳遞。** 例如兩個八位元無號 lane 各自計算 `(255,1)+(1,2)`，截低位結果是 `(0,3)`，不是把整串十六位元當成一個整數相加。

飽和會失去可逆性：127 可由許多超界和得到，無法只靠結果還原輸入。選指令前要確認 lane 寬度、有號解讀與飽和政策。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 乘法：把乘數的每一位變成一列

兩個 n 位元無號數的完整乘積最多需要 2n 位，因為 $(2^n-1)^2 < 2^{2n}$。32 位元輸入的完整乘積最多需 64 位；若只需要低 32 位，也可直接計算模 2^{32} 的結果，硬體不必先形成高半部。

把乘數展開為 $Q = \sum_{k=0}^{n-1} q_k 2^k$，便得到

$$MQ = \sum_{k=0}^{n-1} q_k (M \ll k).$$

q_k=1 就加上左移 k 位的 M，q_k=0 就加零。這些列稱為部分乘積（partial products）。循序硬體逐輪累加，平行硬體先同時生成再壓縮；兩者都要保留各列的位權。

先固定無號或二補數解讀。相同位元在有號乘法中的高位權重不同，不能直接照搬無號部分乘積。

循序乘法：13×11 的四輪狀態

M=`1101`、Q=`1011`、A=0。每輪讀 Q_0，為 1 就做 $C:A = A + M$，否則 C=0、A 不變；再將完整的 (C,A,Q) 邏輯右移一位，C 進入 A 的最高位。

輪次	Q_0	加法結果 $C:A$	右移後 A	右移後 Q
0			`0000`	`1011`
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

最終 `(A,Q)=10001111`$_2$`=143=13×11`。C 保存當輪加法的第五位；不能在右移之前把它丟掉。

循序乘法：用不變量連起每輪狀態

固定初始乘數為 q，完成 k 回合的暫存器為 A_k, Q_k，定義 $R_k = 2^n A_k + Q_k$。精確不變量是

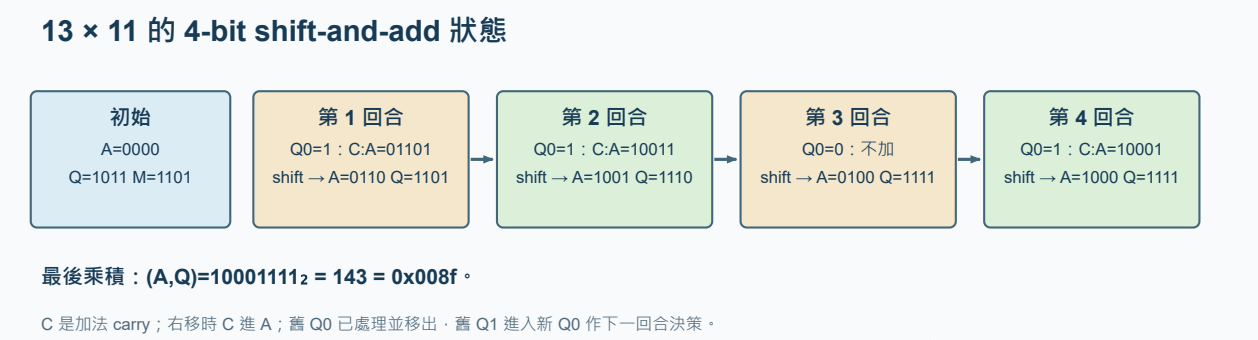
$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)\, 2^{n-k}, \qquad 0 \le k \le n.$$

$k = 0$ 時 $R_0 = q$，成立。若 $k < n$，右項第二項為偶數，所以當前 Q_k 的最低位恰為原乘數第 k 位 q_k。依此位決定加法後，完整 $C : A : Q$ 的值為 $R_k + q_k M 2^n$；右移得到

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1}) 2^{n-k-1}.$$

這使用 $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$，因此歸納延續。$k = n$ 時因 $q < 2^n$，有 $R_n = Mq$；A 為乘積高半部，Q 為低半部。進位 C 保留本回合加法的溢位並隨右移進入 A，沒有遺失該位。13×11的實際 R_k 依序為11、109、158、79、143。

循序乘法：跨暫存器的位元如何移動？



成本模型先寫清楚才可比較：

$$T_{\text{serial}} = t_{\text{setup}} + n\, t_{\text{iter}} + t_{\text{finalize}}.$$

每輪加法與移位若可同週期完成，32 輪才對應 32 個運算週期；若分別占一週期，就需另算。管線化可縮短啟動間隔，單筆延遲仍須依階段數與時脈週期計算。

有號乘法：先決定位元寬度

對兩個 n-bit 二補數輸入，完整乘積放在 $2n$ 個有號位元中。可以先處理大小、最後套用符號，也可以直接用 Booth 重編碼產生帶正負號的部分乘積。

不能在原本的有號寬度內，直接對最小負數取絕對值。 例如四位元二補數的範圍是 $[-8, 7]$，-8 的正大小 8 需要五個有號位元；若改用四位元無號容器，`1000` 才能代表大小 8。

以下先固定 radix-2 Booth：輸入 M, Q 各為 n-bit 有號數；M 稱被乘數，Q 稱乘數。內部將 M 符號擴充至 $n+1$ 位，累加器 A 也用 $n+1$ 位，並額外保存一位 q_{-1}。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Booth 的兩個位元在判斷什麼？

把原乘數的位元寫成 $q_{n-1} \cdots q_0$，補 $q_{-1} = 0$。第 i 個重編碼係數為 $d_i = q_{i-1} - q_i$。本輪讀取順序固定是 **目前最低位在前、上一位在後**：

$Q_0\, q_{-1}$	d	右移前的操作
`00`	0	A 不變
`01`	+1	A ← A + M
`10`	−1	A ← A − M
`11`	0	A 不變

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼重編碼保留二補數的值？

把每個原位元分別收集。最低位之外的正項來自下一個係數，負項來自目前係數；最高位沒有下一項抵消，留下二補數的負權重。

重編碼確實保留二補數的值，因為

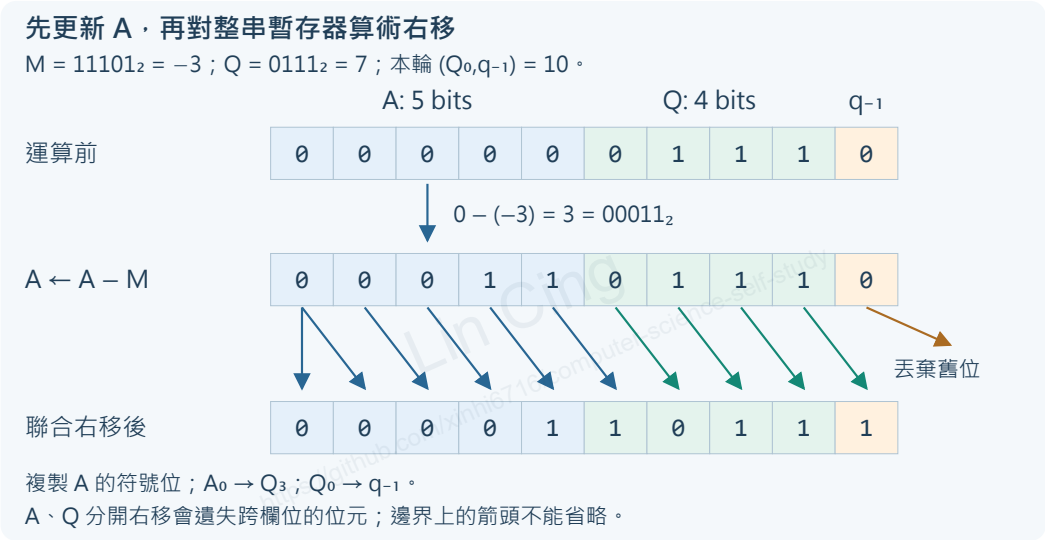
$$\begin{aligned}\sum_{i=0}^{n-1}(q_{i-1}-q_i)2^i &= \sum_{i=0}^{n-2} q_i 2^i - q_{n-1}2^{n-1} \\ &= Q_{\text{signed}}.\end{aligned}$$

中間位元的兩項合成 $(2^{i+1}-2^i)q_i = 2^i q_i$；最高位只留下負權重。這是規則對負乘數也成立的理由。

例如 `0111` 得到低位至高位的係數 −1、0、0、+1，因此值是 −1+8=7；`1101` 得到 −1、+1、−1、0，因此值是 −1+2−4=−3。係數可為負，總和仍是原乘數。

一輪先加減，再聯合算術右移

初始化 $A=0$、$Q=$ 輸入乘數位元、$q_{-1}=0$。每輪先依表更新 A，再將串接的 (A,Q,q_{-1}) **整串算術右移一位**，共做 n 輪。



符號擴充只複製最左端 A 的符號位；A_0 進入 Q 的最高位，舊 Q_0 進入 q_{-1}。結束後丟掉 q_{-1}，(A,Q) 多出的一個最左符號位也是冗餘位；剩下 $2n$ 位就是完整乘積。

−3 乘 7：四輪暫存器追蹤

取 $n=4$，$M=-3$ 擴位為 `11101`；起始 $(A,Q,q_{-1})=$`00000 0111 0`。每列最後才右移，不能用右移後的 pair 決定同一列操作。

輪次	舊 Q_0q_{-1}	A 的操作	加減後 A Q q_{-1}	右移後 A Q q_{-1}
1	`10`	−M	`00011 0111 0`	`00001 1011 1`
2	`11`	0	`00001 1011 1`	`00000 1101 1`
3	`11`	0	`00000 1101 1`	`00000 0110 1`
4	`01`	+M	`11101 0110 1`	`11110 1011 0`

最後 $(A,Q)=$`11110 1011`，是九位元二補數的 -21；移除冗餘符號位得到 `11101011`，八位元值仍為 -21。

原乘數 `0111` 的係數由低位到高位是 $(-1,0,0,+1)$，所以 $7=-1+8$，$(-3)\cdot 7=3-24=-21$。四輪中只有兩次加減，另外兩輪仍須移位。

Booth 數例：正被乘數與負乘數

M = 00010 (+2) Q = 1101 (−3) A = 00000 q_{-1} = 0

輪次	舊 Q_0q_{-1}	操作	算術右移後：A Q q_{-1}
1	10	A − M	11111 0110 1
2	01	A + M	00000 1011 0
3	10	A − M	11111 0101 1
4	11	不變	11111 1010 1

令 $n=4$、$M=2$、$Q=-3$。圖中 A,M 均擴為 5 bits，Q 保持 4 bits；每輪先依舊的 Q_0q_{-1} 選操作，再將整個 (A,Q,q_{-1}) 算術右移。

四輪後 $(A,Q)=$ `11111 1010` 是 9-bit 的 −6；移除冗餘符號位得 8-bit `11111010`，與 $2\times(-3)=-6$ 一致。

多留一個符號位，才能涵蓋最小負數

對 $M=-8, Q=-3$，第一個 pair 是 `10`，必須計算 $A-M=+8$。若 A, M 都只用四位元，`1000` 被解讀成 -8，第一輪便錯了。

−8 × (−3)：取負數之前，先擴充符號位

只用 4-bit A、M：失敗

M = 1000_2 = −8

−M = 1000_2，仍被解讀為 −8

第一次 A−M：1000，不是 +8

右移後：1100 0110 1

四輪結果：11101000_2 = −24

使用 5-bit A、M：正確

M = 11000_2 = −8

−M = 01000_2 = +8

第一次 A−M：01000 = +8

右移後：00100 0110 1

四輪結果：00011000_2 = +24

兩邊都從 A=0、Q=1101_2、q_{-1}=0 開始；差異是加減器與 A 的寬度。

先用五位元精確保存 +8，右移才會得到正確的 +4；四位元路徑卻從 −8 移到 −4。下一頁證明多留一位對所有輸入都足夠。

加減與右移的範圍證明

五位元不是任意補救：每次移位後，A 都落在 $[-2^{n-1}, 2^{n-1}-1]$。加減 n-bit 的 M 後，範圍包含於 $[-2^n, 2^n-1]$，可由 $n+1$ 位精確表示；再右移，便回到前述範圍。因此本模型不依賴中間溢位碰巧抵消。

起始 A=0 符合上述範圍。若某輪開始時 A 已在此範圍，A+M 的最小值是 -2^n，最大值是 2^n-2；A−M 的範圍則是 $[-2^n+1, 2^n-1]$。不操作時也包含於同一個 n+1 位範圍。

算術右移後的新 A，是加減結果除以 2 並向下取整，因此又落回 $[-2^{n-1}, 2^{n-1}-1]$。由歸納法，每輪的加減都能精確執行，包含被乘數等於最小負數的情況。

把暫存器狀態寫成可追蹤的不變量

令 u 是原乘數位元的**無號值**，$d_i = q_{i-1} - q_i$ 仍從原乘數位元定義。做完 k 輪後，令

$$R_k = 2^n \,\mathrm{signed}(A_k) + \mathrm{unsigned}(Q_k), \quad D_k = \sum_{i=0}^{k-1} d_i 2^i.$$

不變量是

$$R_k = \left\lfloor \frac{u}{2^k} \right\rfloor + M 2^{n-k} D_k, \qquad 0 \le k \le n.$$

起始 $R_0 = u$、$D_0 = 0$。對 $k < n$，Q_k 的低 $n-k$ 位尚未被來自 A 的位元覆蓋，故 Q_0 正是原來的 q_k，歷史位則是 q_{k-1}。因此硬體讀到的 pair 確實選出 d_k。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一輪更新保留不變量，最後就是乘積

更新 A 相當於把 $d_k M 2^n$ 加進 R_k；整串算術右移使

$$R_{k+1} = \left\lfloor \frac{R_k + d_k M 2^n}{2} \right\rfloor.$$

代入不變量。當 $k < n$，含 M 的項可被 2 整除，即使它是負數也一樣，因此

$$R_{k+1} = \left\lfloor \frac{u}{2^{k+1}} \right\rfloor + M 2^{n-k-1}(D_k + d_k 2^k).$$

這就是下一輪的形式。做完 n 輪，$0 \le u < 2^n$ 使第一項為零；由重編碼恆等式，

$$R_n = M D_n = M Q_{\mathrm{signed}}.$$

算術右移對負整數相當於向負無窮方向取整，不是向零截斷；這個差別已包含在上面的 floor 中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

減少加減次數，不等於一定減少週期

`0111` 只在 1 的連續區段兩端運算；`0101` 則四個 pair 都是 `10` 或 `01`，需要四次加減。後者的無號 shift-add 只需兩次加法，故 radix-2 Booth 並不對每個輸入都較省。

在本文每輪固定一次控制與移位的模型，兩者都執行 n 輪。是否略過零區段、加減與移位是否同週期、是否管線化，必須另給硬體時序才能換算 latency（單次延遲）或 throughput（吞吐量）。不能直接把「少兩次加法」寫成「快兩倍」。

Radix-4 是另一種重編碼：每次處理兩個新位元，搭配一個重疊的歷史位。它改變部分乘積數量與選取電路，不能只把上圖的右移改成兩位就當作完整規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Radix-4：三位元決定一個帶符號的係數

對偶數 n，第 j 組讀 $(q_{2j+1}, q_{2j}, q_{2j-1})$，其中 $q_{-1} = 0$。相鄰組重疊一位；奇數寬度須先補足符號位。

三位元	係數 c_j	部分乘積的倍數
`000`	0	0
`001`	+1	+M
`010`	+1	+M
`011`	+2	+2M
`100`	−2	−2M
`101`	−1	−M
`110`	−1	−M
`111`	0	0

Radix-4 的係數從哪裡來？

把相鄰兩個 radix-2 係數併為一組，保留低位的權重：

$$\begin{aligned} c_j &= d_{2j} + 2d_{2j+1} \\ &= (q_{2j-1} - q_{2j}) + 2(q_{2j} - q_{2j+1}) \\ &= -2q_{2j+1} + q_{2j} + q_{2j-1}. \end{aligned}$$

所以 $Q = \sum_j c_j 4^j$。三個位元都取 0 或 1，係數只能是 −2、−1、0、1、2。例如 `011` 給 0+1+1=2；`100` 給 −2+0+0=−2；`111` 給 −2+1+1=0。

每往高位移一組，基礎權重乘 4，而非乘 2。最低組補歷史位 0；之後每組共用前一組的最高位。例如 `001111` 由低至高讀成 `110`、`111`、`001`，得到 −1、0、1，因此 15=−1+0×4+1×16。

Radix-4 數例：−7 乘 15

用六位元輸入 $M = -7$、$Q =$`001111`。本頁生成完整十二位元部分乘積；先符號擴充，再依 4^j 左移。

j	位元位置	讀到的三位元	c_j	$c_j \cdot 4^j$	十二位元部分乘積
0	$q_1\ q_0\ q_{-1}$	`110`	−1	+7	`000000000111`
1	$q_3\ q_2\ q_1$	`111`	0	0	`000000000000`
2	$q_5\ q_4\ q_3$	`001`	+1	−112	`111110010000`

$15 = -1 + 0 \cdot 4 + 1 \cdot 16$，所以乘積為 $7 - 112 = -105$，十二位元結果是 `111110010111`。加法在十二位元容器中進行，最高位外的 carry 不屬於結果；其有號值仍是 -105。

Radix-4 的內部寬度要重新分析。 若被乘數是最小負數，$-2M = 2^n$ 需要 $n+2$ 個有號位元。本文先在 $2n$ 位形成部分乘積；沒有宣稱 radix-2 的 $n+1$-bit 累加器可直接套用於所有 radix-4 設計。

乘法加速：先把部分乘積排成同一欄位

以下只處理四位元**無號乘法**：M=13、Q=15。先產生四列部分乘積，每列已依乘數位元的位置左移，全部放在八位元容器中。

列	來源	八位元內容	無號值
P_0	$M \cdot q_0$	00001101	13
P_1	$M \cdot q_1 \cdot 2$	00011010	26
P_2	$M \cdot q_2 \cdot 4$	00110100	52
P_3	$M \cdot q_3 \cdot 8$	01101000	104

目標是把四列的總和 195 算出來。若逐次把兩列加成一個普通二進位數，每次加法都可能等待跨欄進位。Carry-save 的做法是：**中途先保留兩列，最後才把它們加成單一結果**。

三個同權重位元，改存成兩個位元

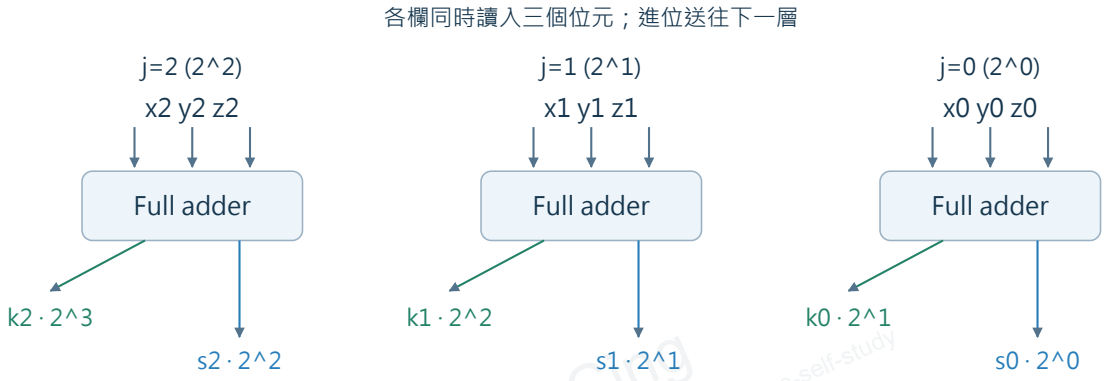
第 j 欄的 x_j、y_j、z_j 都帶權重 2^j。全加器（full adder）只看這三個輸入，輸出同欄的 s_j 與下一欄的 k_j：

$$s_j = x_j \oplus y_j \oplus z_j, \qquad k_j = (x_j y_j) \vee (y_j z_j) \vee (z_j x_j).$$

$x_j+y_j+z_j$	對應輸入	s_j	k_j	恆等式
0	000	0	0	0=0+2・0
1	001、010、100	1	0	1=1+2・0
2	011、101、110	0	1	2=0+2・1
3	111	1	1	3=1+2・1

將各欄恆等式乘上 2^j 再相加，得到 X+Y+Z=S+C，其中 $S = \sum_j s_j 2^j$、$C = \sum_j k_j 2^{j+1}$。**C 已經包含左移一位**，後面不可再移一次。此處的乘法 $x_j y_j$ 是位元 AND，$\vee$ 是 OR。

Carry-save 省下的是「同一層等待進位」

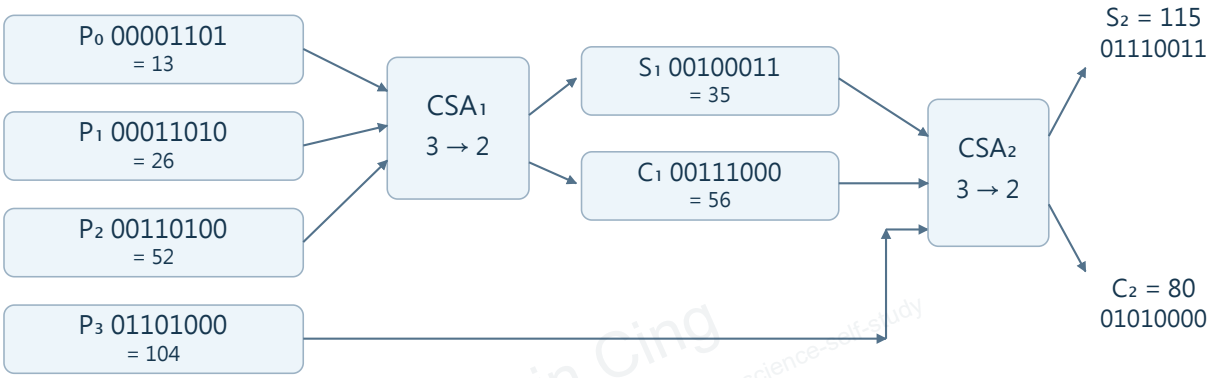


藍色：S 列　綠色：已左移的 C 列；本層全加器之間沒有進位連線

每個全加器都只讀**本層輸入的三列**；它產生的進位寫入下一層的 C 列，不拿來驅動本層左鄰的全加器。圖中 x、y、z 的三個位元可同時備妥，三欄也可並行計算。

這不代表進位消失：它成了另一列資料。S 與 C 各自都還不是總和；只有 S+C 才代表原三列。若每層都立即把 S+C 算完，就重新引入了完整進位傳播。

13×15：四列經兩層壓縮，再相加



總和：13+26+52+104 = 35+56+104 = 115+80 = 195

第一層壓縮 P_0、P_1、P_2，得到 S_1=35、C_1=56，並直接保留 P_3=104；總和仍為 35+56+104=195。第二層得到 S_2=115、C_2=80；兩列相加才得到 195。

圖中的 C_1、C_2 都已按實際權重左移。每一層只壓縮列數，沒有把數值除以 2，也沒有把尚未計入的進位丟掉。

最後的加法，確實還要處理進位

現在做普通加法 $01110011_2 + 01010000_2$。令 c_j 是進入第 j 欄的進位，初始 $c_0=0$；由最低位開始：

j，由低到高	0	1	2	3	4	5	6	7
$S_2[j]$	1	1	0	0	1	1	1	0
$C_2[j]$	0	0	0	0	1	0	1	0
輸入進位 c_j	0	0	0	0	0	1	1	1
結果位	1	1	0	0	0	0	1	1
輸出進位 c_{j+1}	0	0	0	0	1	1	1	0

反向讀結果欄得到 $11000011_2 = 195$。第 4 欄產生的進位一路影響到第 7 欄；這就是最後的 carry-propagate adder（CPA）需要解決的相依。CPA 可用 ripple-carry 或更快的 prefix 結構，不能視為零延遲。

何時可以丟掉最高位外的進位？

對任意三個 W 位元無號數，精確 S 需要 W 位，C 可能需要 W+1 位，而精確總和可能需要 W+2 位。例：W=4、X=Y=Z=15，S=15、C=30，所以總和為 45，不能放進四位元。

若明定只取低 W 位，則可計算

$$X + Y + Z \equiv S + (C \bmod 2^W) \pmod{2^W}.$$

上例四位元 C 只剩 14；15+14=29，再取低四位得到 13，恰等於 45 mod 16。**模數結果正確，不等於精確總和沒有溢位**。

本節無號乘法不同：輸入小於 2^n，所有非負部分乘積的總和小於 2^{2n}。因此每層保存的非負列也不會超過此總和，2n 位足以容納。若使用 Booth 負部分乘積，則須先在指定寬度做二補數與符號擴充；例如前節十二位元的 7+0+3984=3991，低十二位解讀為 −105。此時不能把各列當成原本的無號部分乘積來證明範圍。

列數變少，不等於每一層都是一次完整加法

採用「每層將三列分組壓成兩列，剩餘列直接往下傳」的模型，列數滿足

$$R_{\ell+1} = 2\lfloor R_\ell/3 \rfloor + (R_\ell \bmod 3).$$

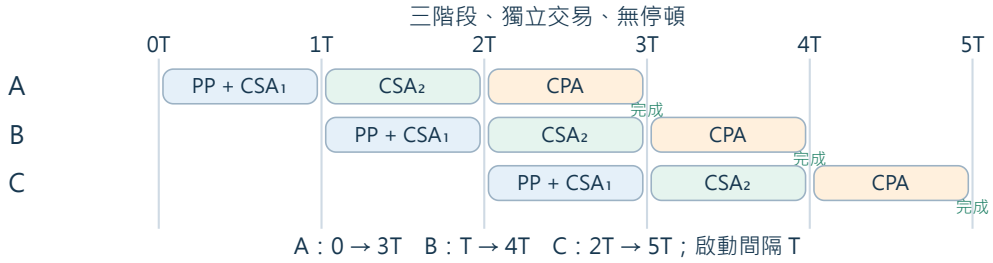
32 列依序變成 22、15、10、7、5、4、3、2，共八層壓縮。這是指定分組模型的層數；實際 Wallace／Dadda 位元堆可逐欄安排全加器與半加器，不能只憑名稱套用此精確數字。

對每個 CSA 層的延遲上界 t_CSA，最長路徑可估為

$$T \le t_{\mathrm{PP}} + 8t_{\mathrm{CSA}} + t_{\mathrm{CPA}} + t_{\mathrm{wire}}.$$

其中 t_PP 包含部分乘積生成；t_wire 明列額外布線成本。八層 CSA 不是八次 W 位 CPA。平衡的二輸入普通加法樹，32 列只需五層，卻每層都可能有進位傳播。兩者延遲要代入電路與負載資料，不能用 8 和 5 直接比快慢。

Pipeline：較常接受新輸入，不代表較早得到同一筆結果



假設把前述四列數例切成三個暫存器階段：①生成部分乘積並完成 CSA₁，②完成 CSA₂，③完成 CPA。每階段都能在共同時脈週期 T 內完成，沒有停頓，且相鄰交易彼此獨立。

交易 A 在邊緣 0 接受、邊緣 3 完成；B 在邊緣 1 接受、邊緣 4 完成。單筆延遲是 3T，啟動間隔（initiation interval）是 T。N≥1 筆獨立交易的最後一筆於 $(N+2)T$ 完成。

若 B 必須使用 A 的乘積，就不能在邊緣 1 當作已取得結果。T 還須容納最慢階段、暫存器與時脈相關成本；未切 pipeline 的電路沒有相同的 T 定義，不能由這張表推論固定三倍加速。

MIPS 乘法：HI 與 LO 分別保存哪一半？

傳統 MIPS32 的 `mult` 以有號方式解讀兩個輸入，`multu` 以無號方式解讀；兩者都產生完整 64 位元乘積，HI 保存高 32 位、LO 保存低 32 位。`mfhi`、`mflo` 再將指定半部搬到一般暫存器。

例如 `0xffffffff×2`：有號解讀為 −1×2，完整結果為 `0xfffffffffffffffe`；無號解讀為 4294967295×2，結果為 `0x00000001fffffffe`。**低半部相同，高半部不同。**

這兩條指令不因乘積超出 32 位元而自動產生有號溢位 trap。若程式要求結果可放入某種 32 位元型別，還須檢查高半部是否符合該型別的延伸規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 除法：先固定商與餘數的規則

無號被除數 D、非零除數 M 要求

$$D = QM + R, \qquad 0 \le R < M.$$

等式與餘數範圍一起決定唯一的 Q、R。只有等式仍不夠，例如 13=2・5+3，也等於 1・5+8；後者不符合餘數範圍。

本節先用恢復餘數除法（restoring division）處理無號輸入，再處理有號的取整規則。除數為零須先排除；有號最小負數除以 −1 則另有可表示範圍問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每輪先帶下一位進來，再試減一次

每輪處理一個被除數位元 d。假設原餘數為 A，先形成 $T = 2A + d$，試算 $T - M$。

試減結果	新商位 b	新餘數 A′
$T - M \geq 0$	1	$T - M$
$T - M < 0$	0	T；若已做減法，就加回 M

已知 $0 \leq A < M$、$d \in \{0, 1\}$，所以 $0 \leq T < 2M$。一次減法就足夠：減或不減之後，兩個分支都回到 $0 \leq A' < M$。這正是每輪只產生一個商位的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13÷5：分開看已讀前綴、商與餘數

由左至右讀取 D=1101。表中的商只包含已產生的位元，不是後面硬體圖的整個 Q 暫存器。

k	新位 d	已讀前綴 P_k	試算 T	T−5	已產生商	新餘數
0	—	0	—	—	空	0
1	1	1	1	−4	0	1
2	1	3	3	−2	00	3
3	0	6	6	1	001	1
4	1	13	3	−2	0010	3

每列都可核對 $P_k = q_k M + A_k$。例如第三列是 6=1・5+1；最後得到 13=2・5+3，且餘數 3 小於除數 5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Q 暫存器同時裝著未讀輸入與已產生的商

硬體初值是 A=`00000`、Q=`1101`、M=`00101`。每輪先將 A、Q 合併左移；舊 Q 的最高位進入 A，Q 的最低位空出來存新商位。

輪次 k	餘數 A（5 位）		Q：未讀輸入 → 已產生商	商 q_k	$P_k=q_k\cdot5+A$
0	00000	= 0	1 1 0 1	0	0
1	00001	= 1	1 0 1 | 0	0	1
2	00011	= 3	0 1 | 0 0	0	3
3	00001	= 1	1 | 0 0 1	1	6
4	00011	= 3	0 0 1 0	2	13

綠：未讀的被除數位元　橘：商位　|　只有 k=4 時，整個 Q 才是商 2。

前 k 輪的數學商 q_k，不等於同一時刻整個 Q 的無號值。 前者只含圖中的商區；後者還混有未讀的被除數位元。到第 n 輪，未讀區才完全消失，Q 才是最終商。

第三輪：跨暫存器移位後，才決定新商位

第二輪結束時 A=3、Q=`0100`。接下來讀的是原被除數第三位 0，故左移形成 T=6，再試減 M=5。

① 移位前

A **00011**

Q **0 1 0 0**

A=3；Q 的最高位是下一位 d=0

→

② A、Q 合併左移

A **00110**

Q **1 0 0 0**

T=2·3+0=6

Q_0 暫填 0

→

③ 試減成功，寫入商位 1

A **00001**

Q **1 0 0 1**

6−5=1 ≥ 0

$A\leftarrow1$ · $Q_0\leftarrow1$

舊 Q_3 進入 A_0；其餘 Q 位元左移。最後一個空位保存本輪商位，不是另一個輸入位元。

圖中試減結果為 +1，寫回 A=`00001`，並把 Q_0 設為 1，得到 Q=`1001`。這個 Q 尚不是最終商 9；最高位的 1 仍是最後一個未讀被除數位元。

一般證明：前綴不變量與餘數範圍

起始 $P_0 = q_0 = A_0 = 0$。假設 $P_k = q_k M + A_k$ 且 $0 \le A_k < M$。下一位為 d，令 $T = 2A_k + d$，選 b=1 當且僅當 T≥M，則

$$P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM).$$

因此 $q_{k+1} = 2q_k + b$、$A_{k+1} = T - bM$，等式持續成立；由 T<2M，餘數也保持在 [0,M)。做完 n 輪後 P_n=D，恰得到要求的商餘數。

本教學實作以 n+1 位保存 T 與試減差：T≤2^n−1，且 −(2^n−1)≤T−M≤2^n−1，皆可精確表示。T 的上界也可由「已讀前綴最多 n 位」與餘數不大於此前綴推出，不能只靠 T<2M 就把任意 T 當成 n+1 位有號數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有號除法：向零截斷與餘數符號

本節固定商向零截斷（truncate toward zero）。先以無號容器計算兩輸入的大小，再依符號還原；非零餘數的符號與被除數相同。

D	M	商 Q	餘數 R	驗算 D=QM+R
7	2	3	1	7=3・2+1
−7	2	−3	−1	−7=(−3)・2−1
7	−2	−3	1	7=(−3)・(−2)+1
−7	−2	3	−1	−7=3・(−2)−1

此時 $|R| < |M|$，但 R 不必非負。若規定商向下取整，−7÷2 改得商 −4、餘數 +1；等式仍成立，規則已不同。零餘數不需要附加正負號條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

邊界與時序：不能只把每輪位數換成倍速

n 位元最小負數的大小 2^{n-1} 可放入 n 位元無號容器，卻不能放入 n 位元正有號數。最小負數除以 −1 的正商超出有號上界，還原符號前就要檢查。

傳統 MIPS32 的 `div`、`divu` 以 LO 保存商、HI 保存餘數；本章版本不因除以零或有號除法溢位自動產生算術 trap，這些輸入下的 HI/LO 結果不可依賴，程式須先檢查。

本節組合暫存器演算法每輪讀一位，共 n 輪；原書先移動寬除數的初版演算法使用 n+1 次，不能混用迴圈數。SRT 等設計每輪可選多位商，但選位、正規化與修正都有成本；每輪兩位不直接證明快兩倍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

算術右移不一定等於有號除法

二補數算術右移一位等於除以 2 後向下取整；本節有號除法則向零截斷。以 −7 為例：

操作	結果	原因
`sra` 一位	−4	$\lfloor -7/2 \rfloor$
向零截斷的除以 2	−3	丟棄小數部分
負數先加 1，再算術右移一位	−3	$\lfloor (-7+1)/2 \rfloor$

一般除以 2^k 可對負數先加 $2^k - 1$ 再算術右移，但必須固定 $1 \le k < n$、輸入型別與不溢位的運算方式。編譯器用移位或倒數乘法替換除法，也必須維持原本的取整規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 浮點格式：範圍與精度分開看

IEEE 二進位浮點格式包含符號、偏移指數與 fraction 欄位。正常值（normal）的有效數首位 1 不存入欄位，因此有效精度比 fraction 多一位。

格式	符號位	指數位	fraction 位	有效精度 p	指數偏移 bias
binary32	1	8	23	24	127
binary64	1	11	52	53	1023

增加指數位主要擴大動態範圍；增加 fraction 位才增加有效精度。欄位指數全零或全一時須改用特殊規則，不能一律補上隱含的 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

浮點欄位：先找到符號、指數與 fraction



圖中的 fraction 不包含正常值隱含的首位 1；欄位位數與有效精度不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

欄位解碼：先分類，再代公式

對 binary32，令 E 是指數欄的無號整數、F 是 23 位 fraction 的無號整數：

E	F	數值或類型
1⋯254	任意	$(-1)^s(1+F/2^{23})2^{E-127}$
0	非零	subnormal：$(-1)^s F 2^{-149}$
0	0	帶符號的零
255	0／非零	無窮大／NaN

subnormal 把正常值的首位 1 改成 0，靠近零的間距固定為 2^{-149}。這是 gradual underflow（漸進下溢），不是把所有過小數直接變成零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

浮點例外：旗標記錄了什麼？

旗標	典型觸發條件	停用 trap 時的例子
invalid	沒有定義的數值操作	0/0、0 • ∞ → quiet NaN
divide-by-zero	有限非零數除以零	1/(+0) → +∞
overflow	以目標精度、無界指數先捨入後超出最大指數	RNE 下最大有限值加自身 → +∞
underflow	本節環境中 tiny 且 inexact	最小正 subnormal 除以 2 → +0
inexact	結果不等於精確值	$1+2^{-24}$ 捨成 binary32 的 1

旗標與程式語言拋出的 exception 不是同一層。是否啟用 trap，以及如何讀取或清除旗標，要由浮點環境指定。後面的數例固定 RNE、停用 trap 與漸進下溢，並另說明 tininess 的判定時點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

浮點加法：先對齊，再決定捨入

有限正常值相加時，先比較指數，把指數較小的有效數（significand）右移到共同尺度；異號輸入改作大小相減，絕對值不同時由較大絕對值的輸入決定符號；大小相等而精確相消時，零的符號依捨入方向決定，例如 RNE 得 +0、朝負無窮方向捨入得 −0。移出的低位仍須保留資訊，不能先截斷。

完成加減後，先正規化，再依目標精度捨入；若捨入產生新的進位，還要再正規化並調整指數。特殊值、零、溢位與 subnormal 路徑須另外處理。

以下使用 **p=4 個有效位（含首位 1）、RNE、指數範圍不設限** 的縮小模型，讓所有中間位元可見；它不是 binary32 格式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對齊數例：sticky bit 如何改變結果

令 a=1、b=9/128=0.0703125。b 原為 $1.001_2 \times 2^{-4}$，對齊到 2^0 後為 0.0001001_2；兩個輸入在 p=4 都是精確值。

p=4（含首位 1），RNE；指數範圍不設限

×2^0	保留 4 bits	G	R	其餘 bits
a = 1	1.000	0	0	00
b = 9/128	0.000	1	0	01
a + b	**1.000**	**1**	**0**	**01**

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

S = OR(0,1) = 1 → GRS = 101 → $1.001_2 = 1.125$

若把 tail 當 00，會誤判為 tie，得到 1.000_2。

精確和為 137/128；相鄰候選是 1 與 9/8，中點為 17/16=136/128。精確和大於中點，故選 9/8，誤差 7/128。若先丟棄低位而回傳 1，誤差是 9/128，已大於半個 ULP=1/16。

相消後的正規化：先找回首位 1

同一個 p=4 模型計算 0.75−0.6875。輸入指數相同，不必先移位；相減後卻要左移三位，才能恢復 normal 的首位 1。

p=4、RNE：0.75 − 0.6875，兩個輸入皆可精確表示

a	1.100 000	× 2^{-1}
− b	1.011 000	× 2^{-1}
相減	0.001 000	× 2^{-1}
左移 3 位	1.000 000	× 2^{-4}

每左移一位，指數減一；數值維持不變。

GRS=000，結果 1/16=0.0625 是精確值，inexact=0。

本例的輸入與差都精確，沒有捨入誤差。相消可能放大**輸入先前已有的相對誤差**，不代表每一次相減本身都不精確。若左移時已碰到最小 normal 指數，必須改走 subnormal 路徑，不能無限降低指數。

Guard、round、sticky：從距離推導 RNE

先看結果的大小。最低保留位記為 L；其後第一位為 guard G，第二位為 round R，更右所有位元取 OR 得 sticky S。S 是「後方是否有非零位」的摘要，不是單一位元的位置。

相對一個保留單位 ULP，G 的權重是 1/2，R 是 1/4，其餘尾端小於 1/4。故 G=0 時低於中點；G=1、R=S=0 時恰在中點；G=1 且 R 或 S 非零時高於中點。中點選最低保留位為 0 的候選，於是

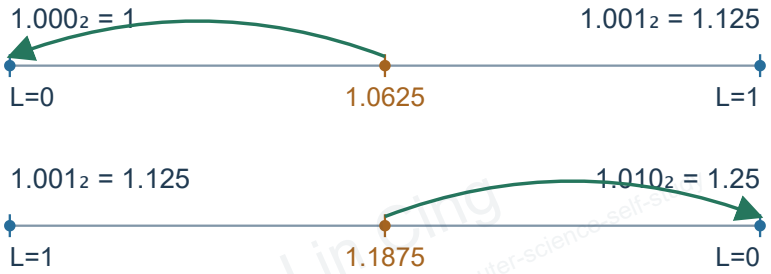
$$\mathrm{inc} = G \wedge (R \vee S \vee L).$$

inc=1 時對**保留的大小**加一個 ULP，之後加回符號。這對負數可能使數值更負，因此不能一律稱為「向上捨入」。只要丟棄部分非零，結果就不等於精確值，須記錄 inexact。

同樣是中點，為何有時保留、有時加一？

下圖仍用 p=4。第一個區間左端為偶，第二個區間右端為偶；even 指有效數整數編碼的最低位是 0，不是十進位值是否為偶數。

p=4、RNE；每條數線只畫一個相鄰區間



inc = G ∧ (R ∨ S ∨ L)

負數先捨入大小，再加回符號：−1.1875 → −1.25。

捨入也可能產生進位。例如 1.1111_2 截成 p=4 後 L=1、GRS=100，須加一：$1.111_2+0.001_2=10.000_2$，再正規化為 $1.000_2\times2^1$。不能只修改 fraction 欄而漏掉指數增加。

Binary32 的兩個捨入檢查點

binary32 在 [1,2) 的間距為 2^{-23}，精度 p=24。對 $1+2^{-24}$，L=0、GRS=100，故選 1（0x3f800000）；對 $1+3\cdot2^{-25}$，GRS=110，故選 $1+2^{-23}$（0x3f800001）。兩次皆不精確，inexact=1。

Binary32 round-to-nearest-even 與五種 FP 例外

Tie：$1+2^{-24}$	Above half：$1+3\cdot2^{-25}$	Underflow tie：2^{-150}	Overflow：max+max
GRS=100，保留 LSB=0	GRS=110	0 與 2^{-149} 的中點	exact result > max finite
candidate：1.0	round up：$1+2^{-23}$	even→+0	rounded→+∞
inexact	inexact	underflow + inexact	overflow + inexact

Invalid：0×∞、∞−∞、0/0 → quiet NaN；1/±0 → signed infinity。

G/R/S 是 guard、round、sticky；tie 選最低保留 bit 為 0，NaN 依 payload 與 comparison 規則傳播。

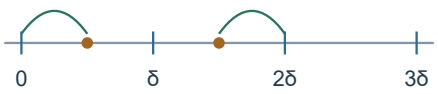
這裡的 2^{-24} 是相鄰值間距的一半；不是「精確和的一半」。FMA 對精確 ab+c 只捨入一次，分開的乘法與加法各自捨入，結果可能不同；本章稍後給出可核對的位元數例。

Subnormal：範圍延伸與旗標要分開判斷

令 $\delta=2^{-149}$。binary32 的正 subnormal 為 $F\delta$，其中 fraction 整數 $1\le F\le 2^{23}-1$；最小 normal 為 $2^{23}\delta=2^{-126}$。接近零的間距固定為 δ，跨到最小 normal 時仍連續。

binary32: $\delta = 2^{-149}$; RNE; gradual underflow

靠近 0

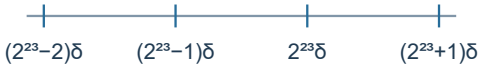


$\delta/2 \to 0$; $3\delta/2 \to 2\delta$

underflow=1, inexact=1

精確 δ：兩個旗標都為 0

靠近最小 normal



0x007fffff = $(2^{23}-1)\delta$

0x00800000 = $2^{23}\delta$

跨邊界的間距仍為 δ

右圖第一點的 index 是 $2^{23}-2$；兩個視窗之間不採共同原點。

Subnormal 不等於一定設定 underflow 旗標。 精確的 $2^{-126}/2$ 得 0x00400000，沒有誤差；圖中兩個中點則因 tiny 且 inexact，同時設定兩個旗標。下一頁明定浮點環境並列出比較。

用同一張表核對 subnormal 的精確性

本表固定 RNE、gradual underflow、停用 trap、旗標初值為 0，並在捨入後判定 tininess。所有輸入皆能以 binary32 精確表示；UF、NX 分別代表 underflow 與 inexact 旗標。

運算	精確值	RNE 結果 bits	UF	NX
$2^{-126}/2$	2^{-127}	0x00400000	0	0
$\delta/2$	$\delta/2$	0x00000000	1	1
$(3\delta)/2$	$3\delta/2$	0x00000002	1	1
最小 normal − 最大 subnormal	δ	0x00000001	0	0

前兩個中點以 fraction 整數的奇偶選擇：0 與 1 之間選 0；1 與 2 之間選 2。旗標不儲存誤差大小；通常新觸發的旗標會累積，直到明確清除。若要判斷單次運算是否觸發，應先清除舊旗標。若開啟 flush-to-zero（FTZ）或把 subnormal 輸入當成 0，已改變本節模型，不能直接套用此表。

表示法與 gradual underflow 可參照 Oracle Numerical Computation Guide § 2.3。

特殊值：同時寫出結果與例外

最大有限值加自身在 RNE 下得 +∞，設定 overflow 與 inexact。0×∞、∞−∞、0/0 為 invalid，停用 trap 時回傳 quiet NaN。

有限非零數除以 ±0 得帶符號的 infinity，符號由兩輸入的 sign bits XOR 決定，設定 divide-by-zero；∞/+0 則仍為 infinity，不因這次運算設定 divide-by-zero。詳見 Oracle Numerical Computation Guide 的例外表。

NaN 的比較須依 predicate 判斷：普通相等比較即使兩邊都是同一個 NaN 也為 false，不等比較為 true；ordered 與 unordered 不能混用。+0 與 −0 比較相等，但倒數與 signbit 可區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

浮點乘法：符號、指數、有效數各做什麼？

先限於有限正常輸入 $a=(-1)^{s_a}m_a2^{e_a}$、$b=(-1)^{s_b}m_b2^{e_b}$，其中 $1\le m_a,m_b<2$。符號為 $s_a\oplus s_b$，精確有效數乘積在 [1,4)，未正規化指數為 e_a+e_β。

若乘積≥2，右移一位並把指數加一；再依目標精度捨入，必要時再次處理進位。例 $1.5\times1.5=10.01_2=1.001_2\times2^1$，正規化需要右移一位，不能寫成固定左移。

儲存的指數有 bias，兩個指數欄相加時要扣回一次 bias，再加入正規化調整。subnormal、零與無窮大另外分類；0 • ∞ 屬 invalid，不能走一般「零乘任何數得零」路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIPS 浮點：資料格式也寫在指令中

本章傳統 MIPS 的浮點暫存器位於 coprocessor 1，與一般整數暫存器分開。`lwc1`、`swc1` 搬移一個浮點 word 的位元；`add.s` 與 `add.d` 則選擇不同浮點格式的加法。

浮點比較設定條件碼，再由 `bc1t`、`bc1f` 分支。遇到 NaN 時，必須確認使用的是哪種 ordered／unordered 判斷，不能把它當成普通整數比較。

同一串位元可被搬移而不做數值轉換；要改變格式或轉成整數，須使用相應轉換指令。暫存器配置與保存區大小也要配合格式及 ABI，不能只看變數名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

捨入方向與誤差尺度

RNE（round to nearest, ties to even）選最近值，中點才看最低保留位。方向性捨入則可朝 $+\infty$、$-\infty$ 或零；它們回答不同需求，例如區間上下界不能任意用 RNE 代替。

ULP 描述某個尺度附近相鄰可表示值的間距。binary32 在 [1,2) 的間距為 2^{-23}，在 [2,4) 則為 2^{-22}。同一個絕對誤差，在接近零與遠離零時的相對大小不同。

有效精度 p 控制可保留的有效二進位位數，指數範圍控制能表示多大或多小。兩者分別限制細節與尺度，不能用「位元更多」概括。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NaN 與帶符號的零：不能照整數排序

普通相等比較中，NaN 與任何值都不相等，包括自身；不等比較則為真。要辨識 NaN，使用 `isnan` 或指定的 unordered 判斷，並確認比較指令的例外規則。

IEEE 相等比較中 +0=−0，但符號位不同。倒數可分成 +∞、−∞，`signbit` 也能區分，因此相等不代表編碼相同。

浮點負數的編碼順序、帶符號的零與 NaN 都使「把位元當有號整數直接比較」失去一般正確性。應使用對應格式的浮點比較規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 平行求和：結果為什麼會隨順序改變？

精確整數加法，以及固定模數的加法，都有結合律；逐步捨入的浮點加法則可能使 $(a+b)+c \neq a+(b+c)$。平行歸約（parallel reduction）改變求和樹，也就可能改變中間捨入。

若要求位元完全重現，須固定求和次序，以及格式、捨入模式、中間精度與 FMA 收縮等環境；或採用有明確順序無關保證的累加器。補償求和與提高精度主要改善誤差，本身不保證換序後位元相同。

因此結果略有差異時，先核對運算順序與浮點環境；不能只憑差異就判定有資料競爭，也不能因此排除真的競爭問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

補償求和也不是順序無關的保證

固定標準 Kahan 的每次基本運算均以 binary32、RNE 執行。其更新為 y=a−c、t=s+y、c=(t−s)−y、s=t；起始 s=c=0。

讀入順序	關鍵中間步驟	最後 s
$[2^{25}, 1, -2^{25}]$	加 1 時 s 不變，c=−1；下一步 $-2^{25}+1$ 又在中點捨回 -2^{25}	0
$[2^{25}, -2^{25}, 1]$	前兩項先精確抵銷，再加 1	1

補償變數能保存某些丟失資訊，但後續計算它或使用它時仍會捨入。這個數例否定「補償求和一律能保證換序結果相同」，不否定它在許多資料上改善誤差的價值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

換結合順序，就換了捨入時點

運算順序的影響可用確切位元驗證。以 binary32 $x = 2^{24}$ 為例，x 的 ULP 是 2。x+1 恰好落在 x 與 x+2 的中點，最低保留 bit 要求選 even，所以先 round 成 x：

$$((x + 1) - x) = 0, \qquad (x - x) + 1 = 1.$$

兩式在實數算術中相等，逐次 binary32 捨入卻不同；差異來自捨入時點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FMA：精確乘加後只捨入一次

取 $a = b = 1 + 2^{-23}$（`0x3f800001`）、$c = -(1 + 2^{-22})$（`0xbf800002`），皆能以 binary32 精確表示。

$$ab = 1 + 2^{-22} + 2^{-46}, \qquad ab + c = 2^{-46}.$$

計算路徑	中間捨入	最終結果
先乘再加	乘積先捨成 $1 + 2^{-22}$，小項消失	+0
FMA	精確形成 ab+c 後只捨入一次	2^{-46}，`0x28800000`

差別來自捨入時點，不是實數乘法規則改變。本章傳統 MIPS 核心不保證提供 FMA；使用相應擴充時，仍要指定運算元格式與捨入模式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7 中間精度：暫存器裡算完，存回後可能不同

本章歷史案例比較 x87 的延伸精度堆疊暫存器與 SSE2 的 XMM 暫存器。若中間值先留在較寬格式，與每步都捨入到 binary32，最後結果可能不同。

例如 binary32 的 $1 + 2^{-24}$ 恰為中點，RNE 先捨成 1，再減 1 得 0；若中間格式能精確保留該和，再減 1，可得 2^{-24}。若較早存回 binary32，差異又會消失。

重現數值結果須記錄格式、捨入模式、中間精度、存回時點與編譯選項。此歷史比較不代表所有 x86 程式都固定使用同一種中間精度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.8 三個常見推論，如何用反例檢查？

常見推論	本章反例或缺少的條件
有進位就有有號溢位	`0xff+1` 有進位，二補數 −1+1 卻精確為 0
每輪處理兩位就一定快兩倍	每輪延遲、前置處理及修正次數可能增加
浮點是近似，所以順序不重要	binary32 的 $(2^{24}+1)-2^{24}=0$，換序可得 1

編譯器的 fast-math 選項可能允許重排、改變結合順序或假設沒有特殊值。若程式依賴特定旗標、NaN 或逐步捨入結果，就要先確認允許的轉換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.9 一個結果，至少要留下三種證據

檢查層次	應留下的證據	常見漏項
表示	輸入寬度、型別與解碼值	把二補數當無號數
運算	中間狀態、範圍與一般不變量	只有最後答案，沒有進位或餘數
交付結果	截取／捨入、例外、寫回政策	精確數學值不在輸出格式中

乘除法另須區分回合數、單筆延遲與啟動間隔。浮點數另須檢查正常值、零、subnormal、無窮大、NaN 及中點；只測普通正數不足以驗收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

格式標準化之後，仍有哪些問題？

IEEE 浮點格式讓位元解碼、捨入與特殊值有共同規則；乘法器的重編碼、部分乘積壓縮、管線化與 SIMD 則改善不同層次的硬體成本。

格式標準不能替代演算法的數值分析。病態問題可能放大輸入誤差，求和樹改變可能改變捨入結果，多執行緒也可能改變運算順序。

因此閱讀硬體效能宣稱時，分別問：使用什麼格式、每次運算遵守什麼規則、單筆與連續多筆花多少時間、整個演算法的誤差如何累積。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

讀完後，試著完成這些檢查

1. 從輸入符號與進位，分別判斷有號及無號加法溢位。
2. 逐輪追蹤乘除法，說明暫存器寬度與一般不變量。
3. 編碼、解碼 binary32，區分正常值、subnormal、無窮大、NaN 與帶符號的零。
4. 從對齊後的 guard、round、sticky 位元推導 RNE 結果。
5. 對五種浮點例外分別寫出結果與旗標，包含精確 subnormal 的反例。
6. 用確切數值說明換結合順序及 FMA 如何改變逐步捨入結果。
7. 在指定時序模型下，分別計算單筆延遲與穩定吞吐量。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 3. CSO course slides, arithmetic.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study